Cursor Workshop

Forta Solutions

January 20, 2025

About Us

Rogier Muller

- CTO at a BlueMonks Group (FinTech)
- Founder of a B2B Agency focused on developer-efficiency
- Cursor Ambassador for the Netherlands

Vasilis Tsolis

- Founder of Cognitiv+
- Expertise in AI-assisted development workflows
- Cursor Ambassador for Greece

Agenda

- Welcome and Setup Check
- What’s New in Cursor 2.0 / 2.2
- Context Management
- Methodology (Delegate, Review, Own)

Break

- The Codebase: Aegistry
- Feature Introduction
- Hands-on Labs: Plan & Build Phase
- Day 1 wrap-up

Opening thoughts

Pratham
@Prathkum

Most of coding was never about writing code.

AI is just making this more obvious.

You no longer need to recall syntax, function structure, boilerplate code, or even API endpoints.

That's the easy part and AI is very good at it.

The hard part was never typing. It was always thinking.

And it still is.

9:12 AM · Dec 9, 2025 · **500.6K** Views

Setup Check

- Latest version of Cursor (2.3.35, as of Jan 18th)
- Repo downloaded/cloned
- Credits available (show of hands)

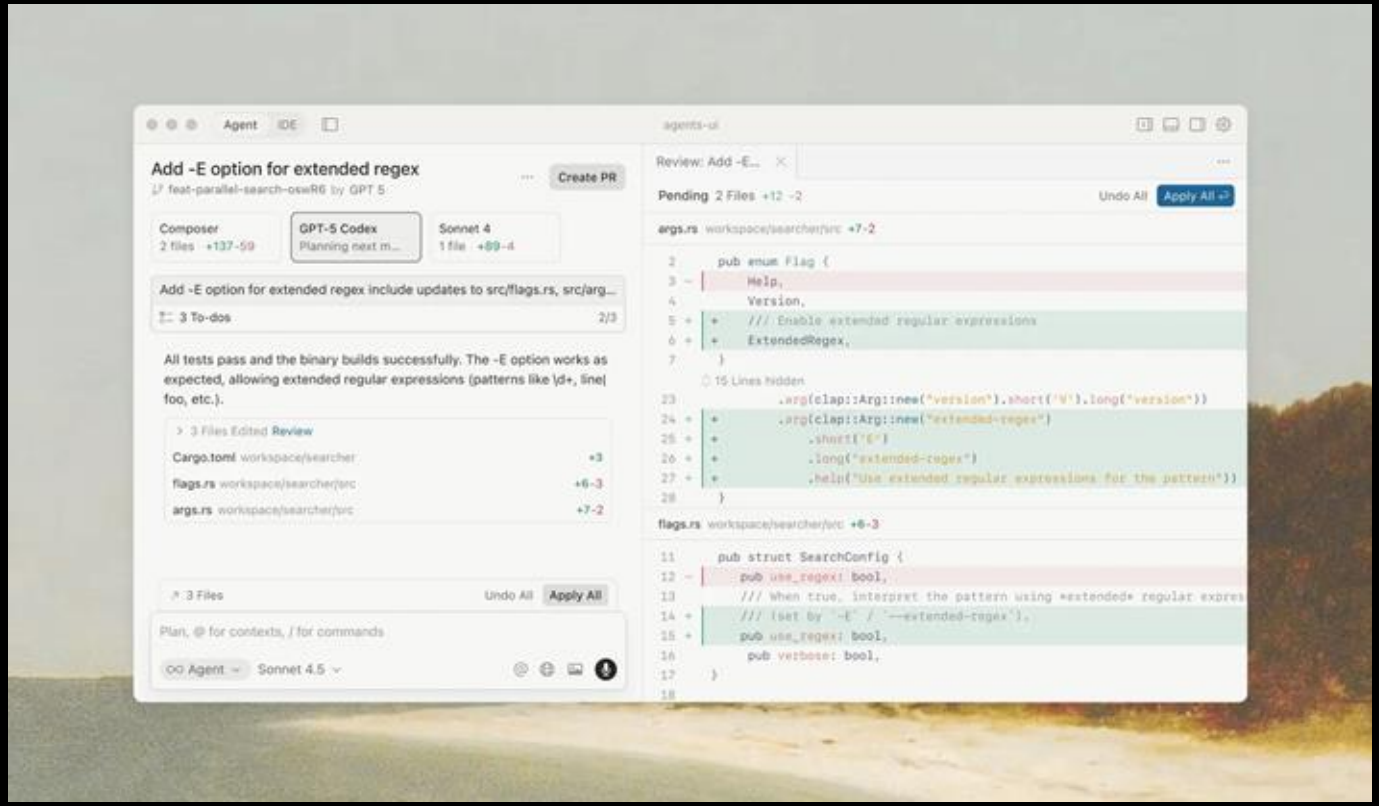
What is Cursor?

- Visual Studio Code + Agentic AI
  - Cursor combines the power of VS Code with AI assistance built right in
- Files on Your Computer
  - Unlike ChatGPT or Claude in the cloud, Cursor works with your local files
- Switch Models Mid-Conversation
  - Choose the right AI model for each task - planning vs execution

What’s new in Cursor 2.0 (October 29th)

- Multi-agent interface for planning, UI & backend

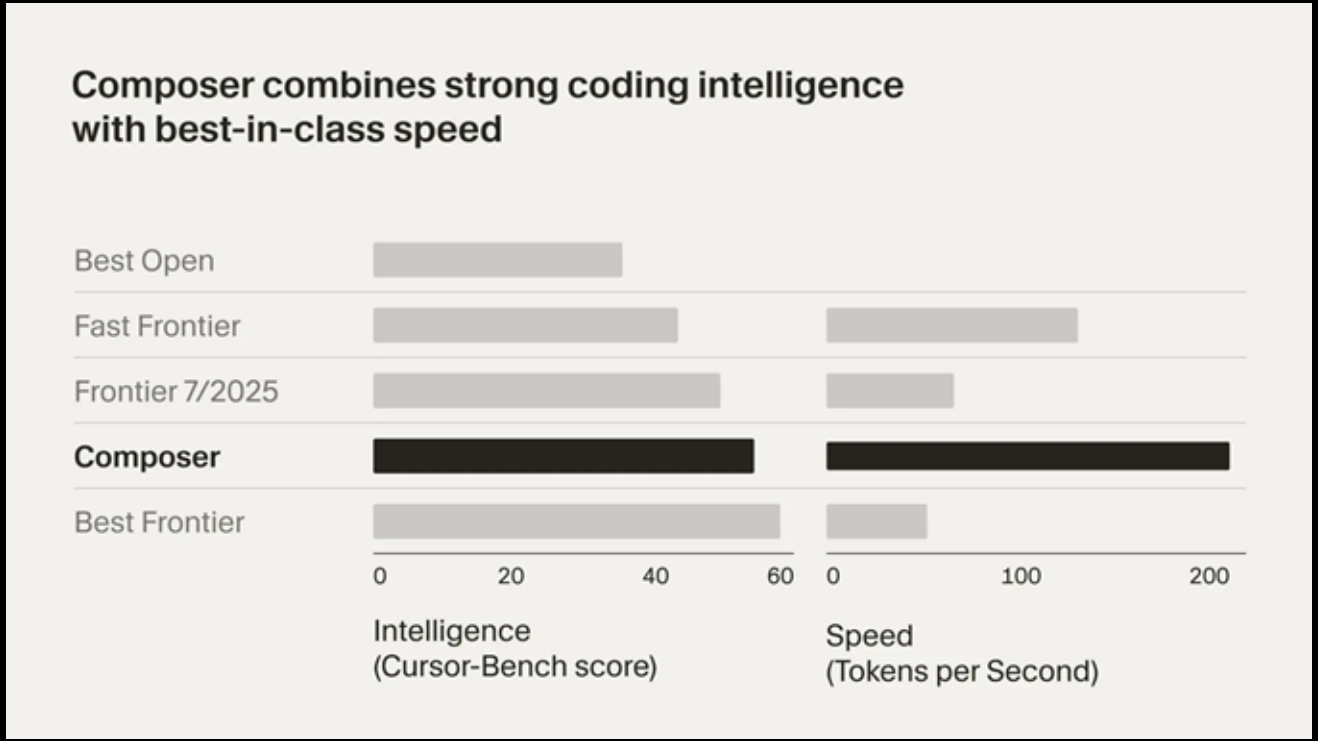
What’s new in Cursor 2.0 (October 29th)



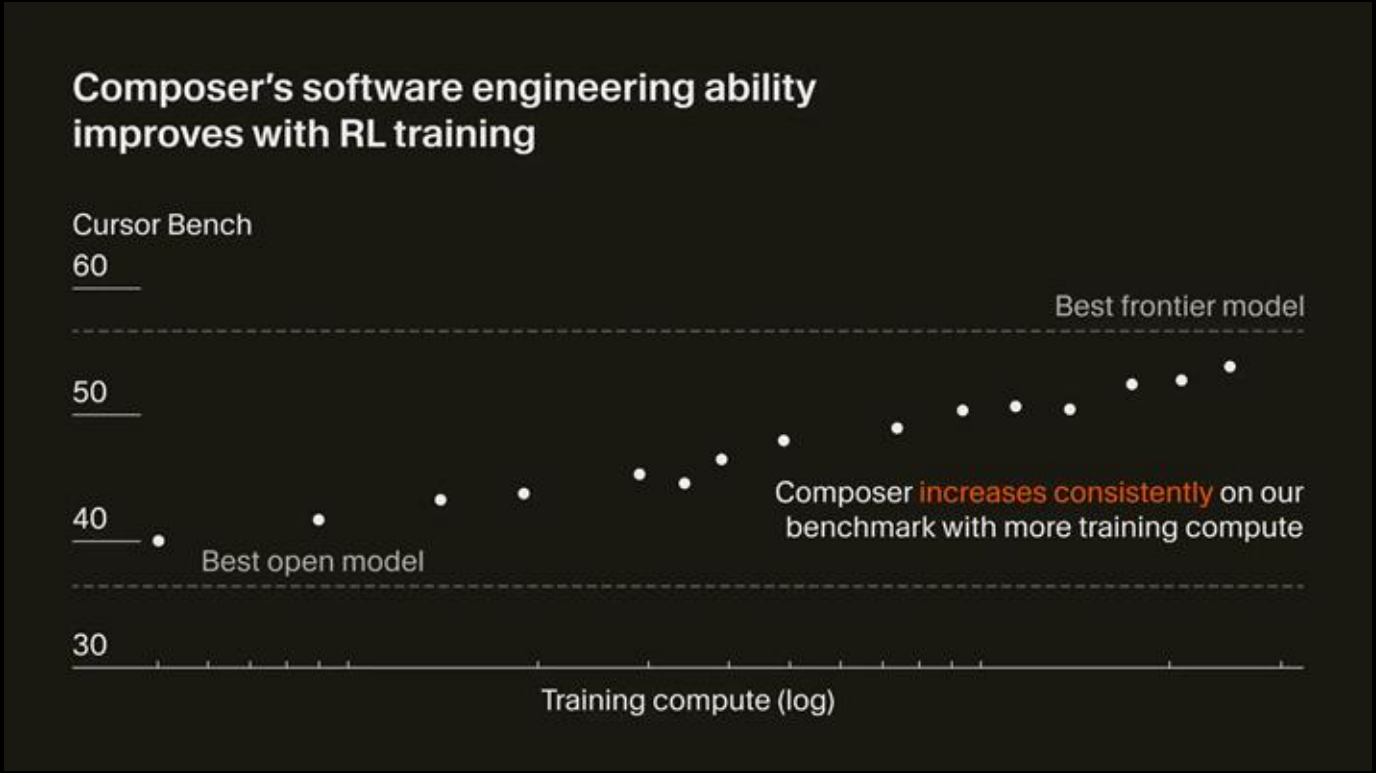
What’s new in Cursor 2.0 (October 29th)

- Multi-agent interface for planning, UI & backend
- Composer-1: fast, coding-first model

What’s new in Cursor 2.0 (October 29th)



What’s new in Cursor 2.0 (October 29th)



What's new in Cursor 2.0 (October 29th)

Composer learns to be a more effective and more efficient agent

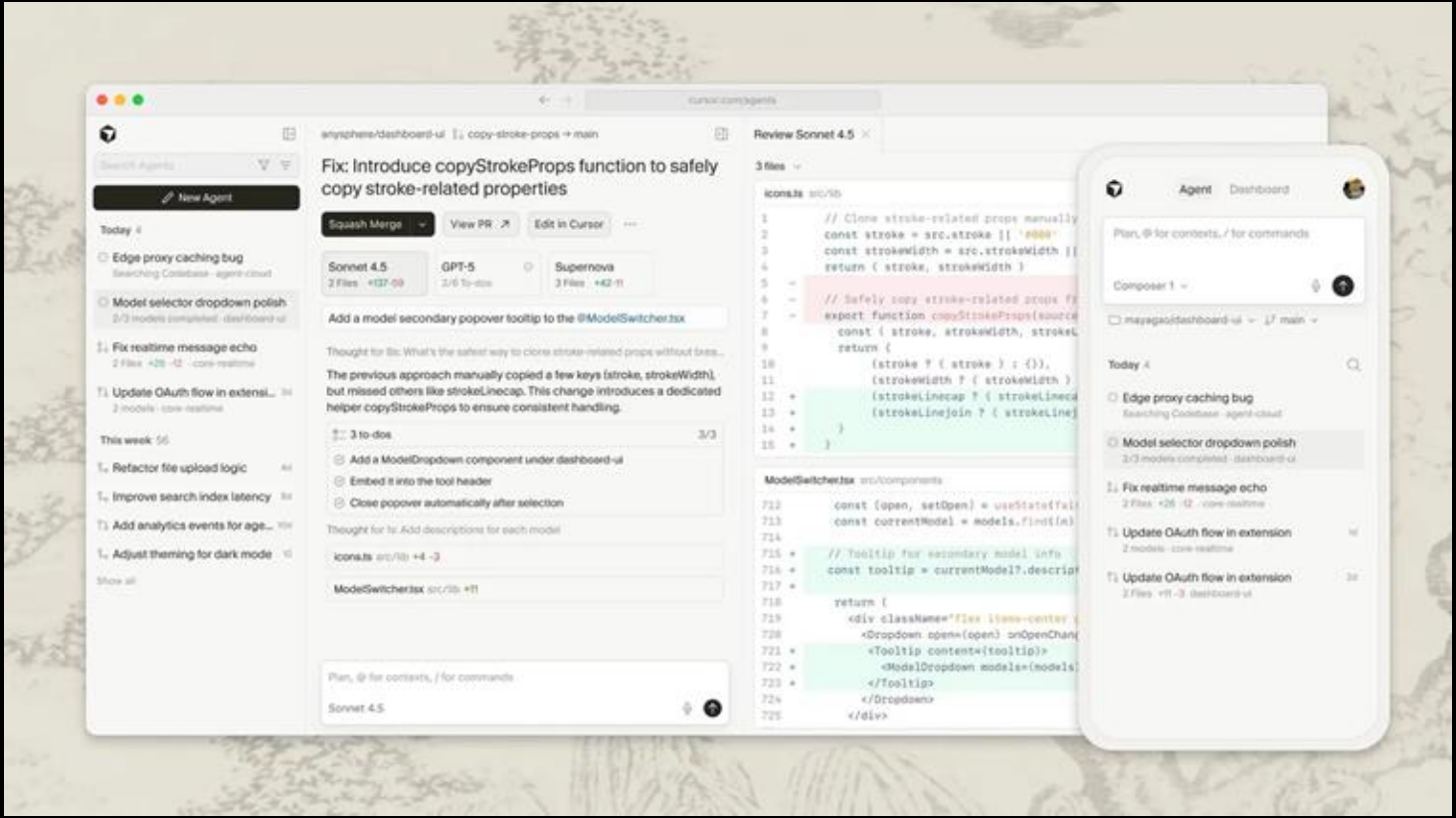
Composer learns to utilize parallel tool calls for more efficient tool usage.

Composer uses more search and file read tools relative to code updates.

What’s new in Cursor 2.0 (October 29th)

- Multi-agent interface for planning, UI & backend
- Composer-1: fast, coding-first model
- Cloud Agents: Async. reliable longer-running jobs

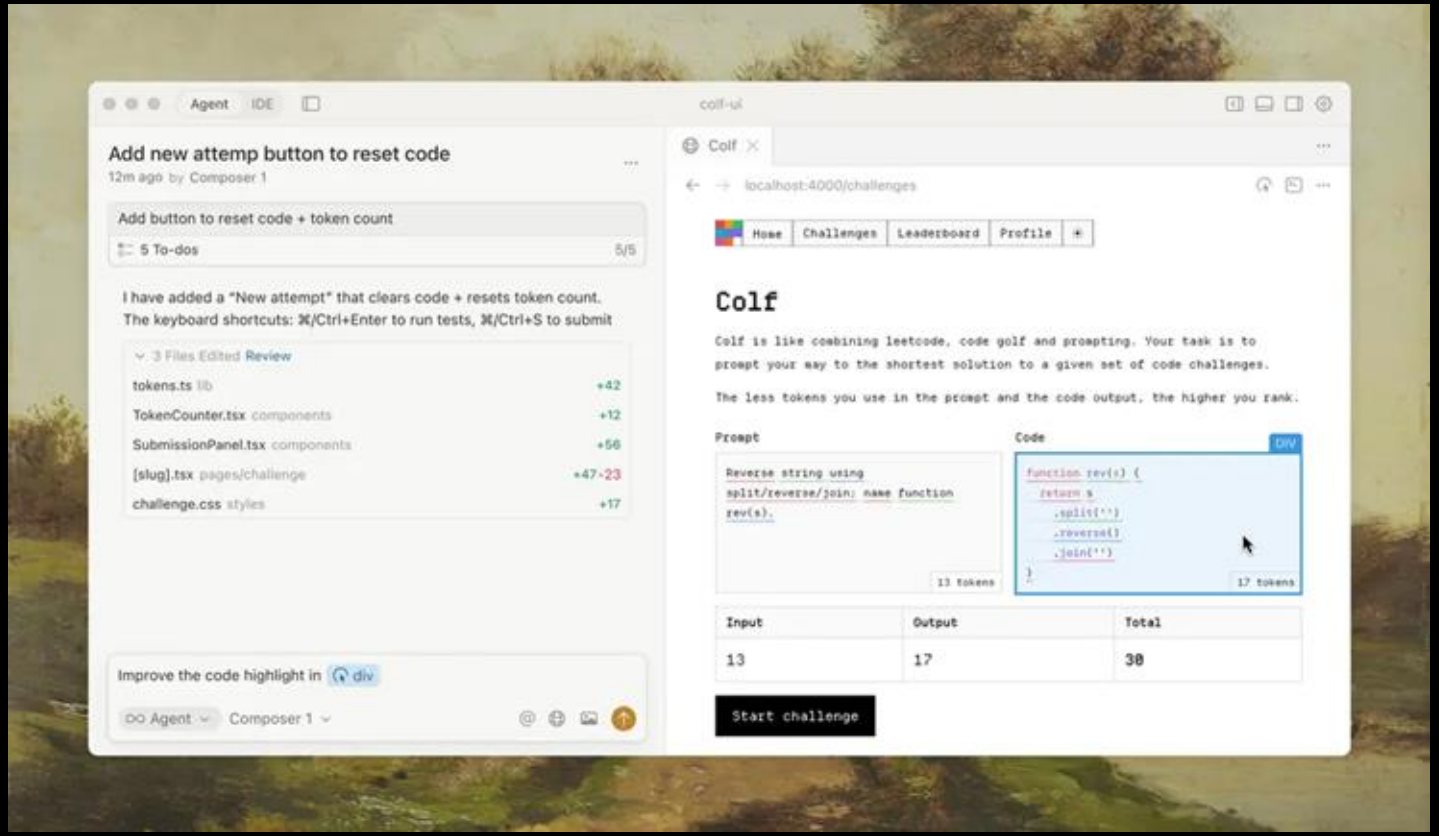
What’s new in Cursor 2.0 (October 29th)



What’s new in Cursor 2.0 (October 29th)

- Multi-agent interface for planning, UI & backend
- Composer-1: fast, coding-first model
- Cloud Agents: Async. reliable longer-running jobs
- In-app Browser & sandboxed Terminals (GA)

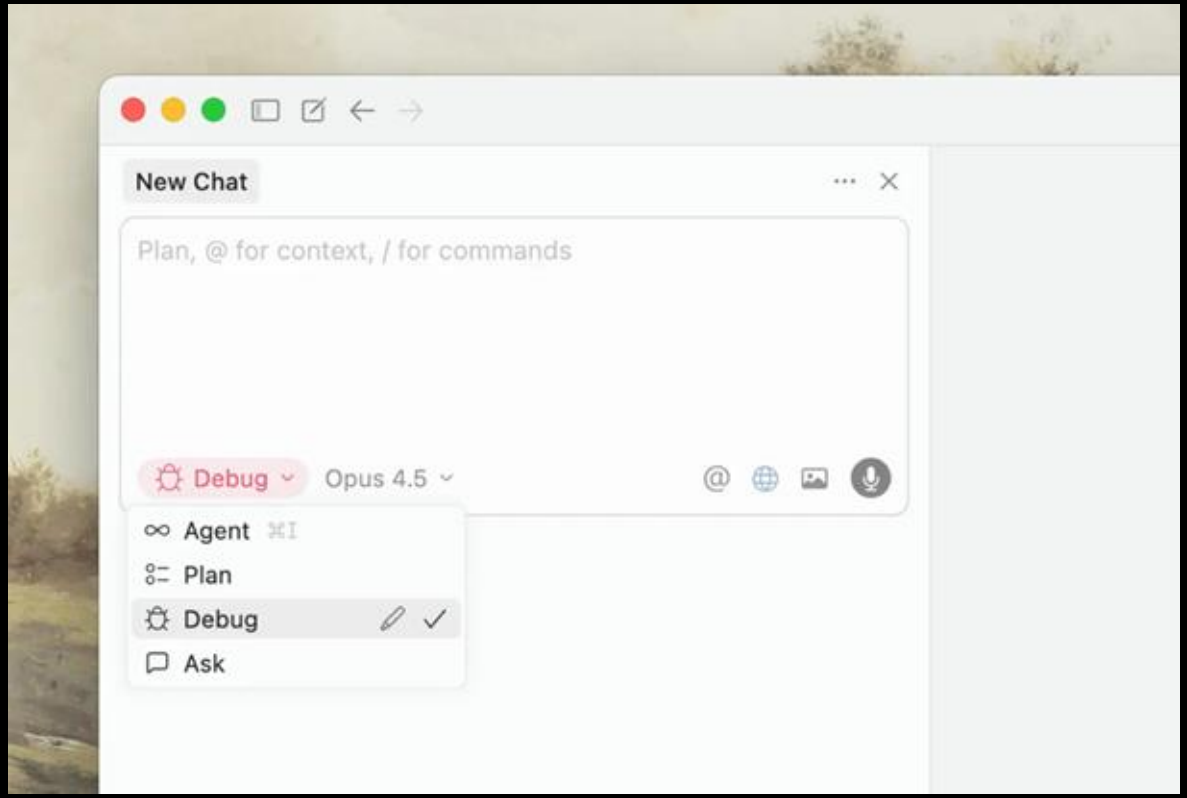
What’s new in Cursor 2.0 (October 29th)



What’s new in Cursor 2.2 (December 10th)

- Debug-mode

What’s new in Cursor 2.2 (December 10th)



What’s new in Cursor 2.2 (December 10th)

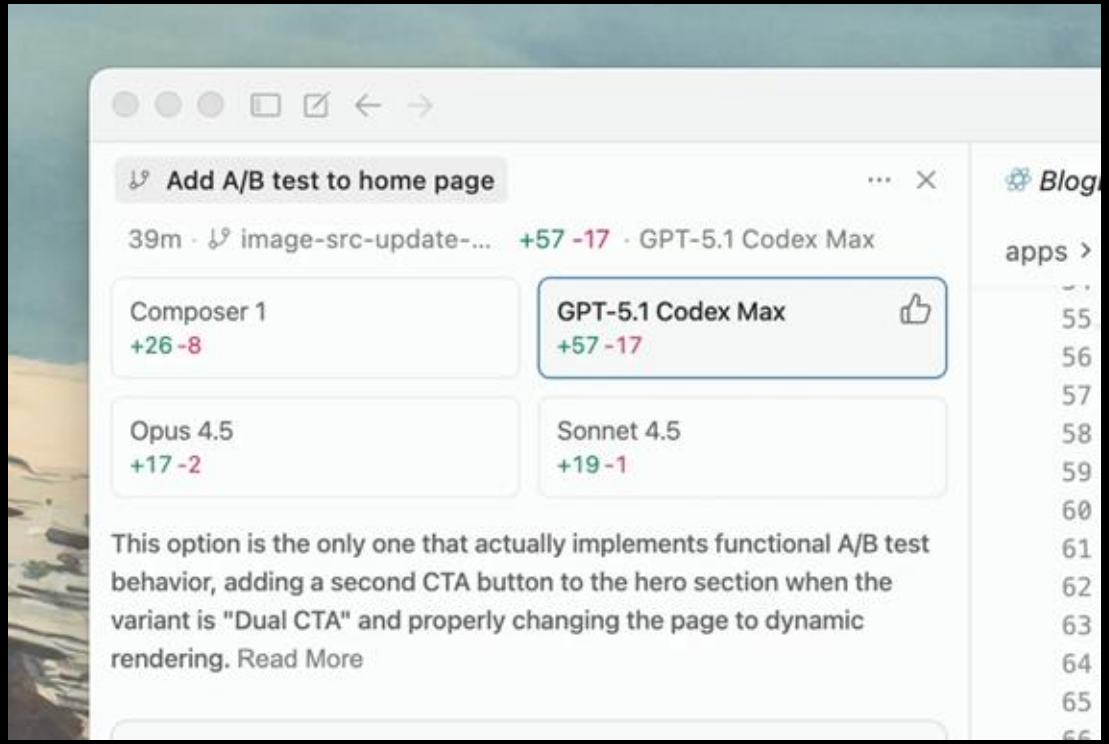
- Debug-mode
- Plan Mode improvements

What’s new in Cursor 2.2 (December 10th)

What’s new in Cursor 2.2 (December 10th)

- Debug-mode
- Plan Mode improvements
- Multi-agent judging (the LLM council)

What’s new in Cursor 2.2 (December 10th)



Context Management

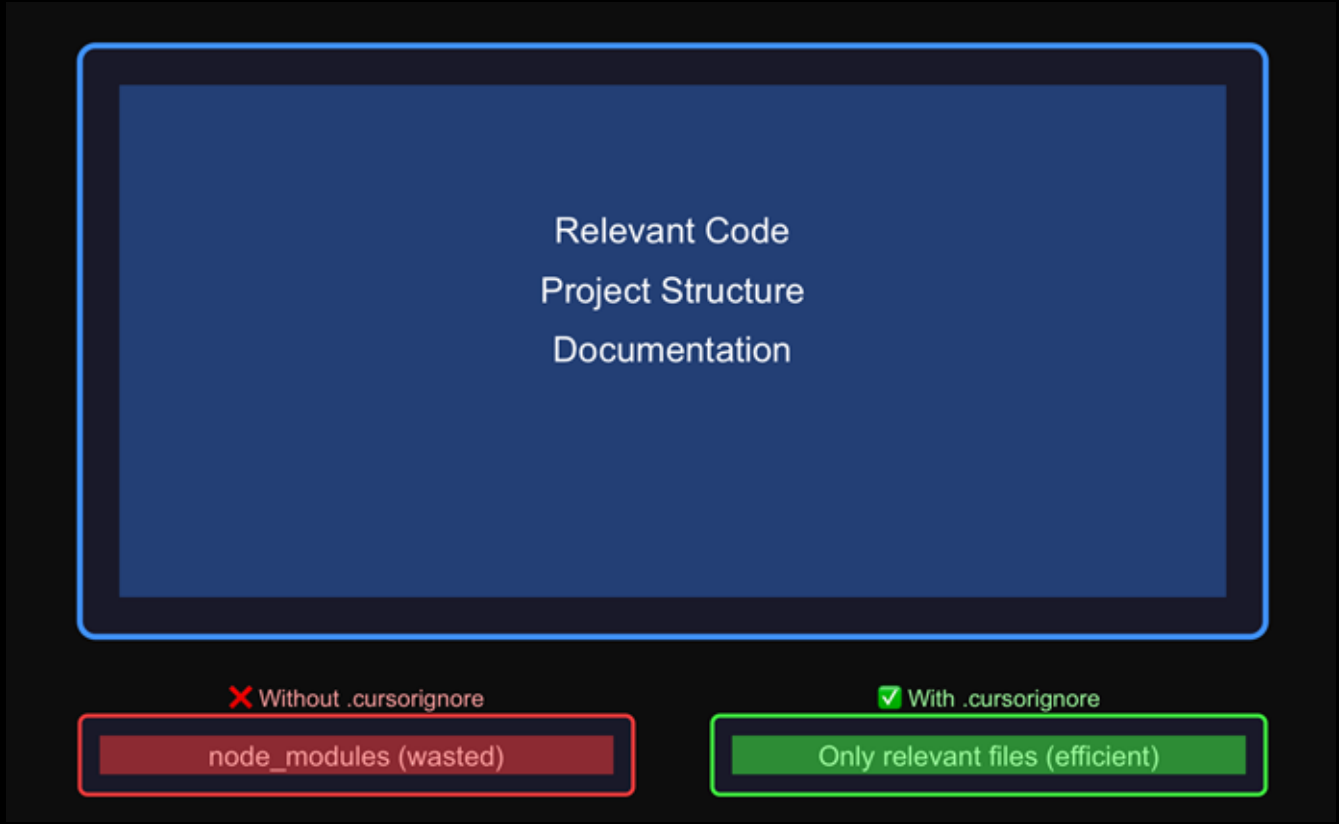
When working with AI assistants:

- AI doesn't remember previous sessions
- Large codebases can't fit entirely in context (200k–1m)
- Team members (and AI) need to understand what changed

The Solution: Three Critical Files

- AGENTS.md – The Constitution for AI
- CHANGELOG.md – Source of Truth for progress
- .cursorignore – Everything Cursor needs to ignore

Context Management



Documentation & Markdown

Why Create Documentation in Markdown?

- AI needs context to help you effectively. Documentation provides that context.
- Human-readable format
- Structured and easy to parse
- AI understands context better
- Works perfectly with Cursor's @file references

Markdown Format Basics:

```
# Heading
## Subheading

- Bullet point
- Another point

**Bold text** and *italic text*

`code` and code blocks
```

Context Management: AGENTS.md

Location: /AGENTS.md (replaces .cursorrules (deprecated))

Purpose:

- Operating principles for AI assistants
- Documentation rules (what files to use, what to avoid)
- Code style preferences
- Decision-making guidelines

Key Rules:

- NEVER create additional docs
- Always update changelog after changes
- Keep replies concise

Context Management: CHANGELOG.md

Purpose: Track all changes in reverse chronological order

What to Record:

- What changed
- What files were impacted
- Why the change was made

Format Example:

- ## 15-01-2026
- - Added Risk Score Audit Trail feature
- - New endpoints: GET /v1/audit, GET /v1/audit/:request_id
- - Files: audit.rs, tenant_db.rs, main.rs

Context Management: CHANGELOG.md (cont.)

- Add strict instructions in AGENTS.md **how to shape** CHANGELOG.md

```
## CHANGELOG PROTOCOL

For **every change** (large or small), add a single line to `docs/CHANGELOG.md` with:
- **a)** What changed
- **b)** What files, variables, classes impacted
- **c)** How this solved a specific problem

**Format:** NEWEST ENTRIES ON TOP. Use `## DD-MM-YYYY` subheading for each date.
```

Context Management: Context Recovery

When starting a new session:

> “View @docs/changelog.md @docs/blueprint.md, and the commit history I'm starting a new session. What was recently added? What should I know before making changes?"

Why this matters:

- AI immediately has context
- No need to re-explain the project
- Consistent understanding across team members

Context Management: Best Practices

Keep It Updated

- Update changelog after EVERY feature or fix
- Review AGENTS.md monthly (and share it within the team)

Keep It Concise

- AGENTS.md: 100-200 LOC max
- Changelog: Essential info only (force the agent to adhere to strict rules)

Keep It Enforced

- Add to PR checklist
- AI should refuse to work without reading these files

Methodology: The 7 Phases

1. Plan - Define what to build
2. Design - Visual and technical design
3. Build and Deploy - Write code and ship
4. Test - Verify correctness
5. Review - Security and quality check
6. Document - Update AGENTS.md and changelog
7. Deploy and Maintain - Operations and monitoring

Each phase follows: Delegate → Review → Own

Methodology: Delegate, Review, Own

Delegate – What AI Does Well

- Generate code, scan repos, analyze dependencies
- Multi-file edits, boilerplate, refactoring

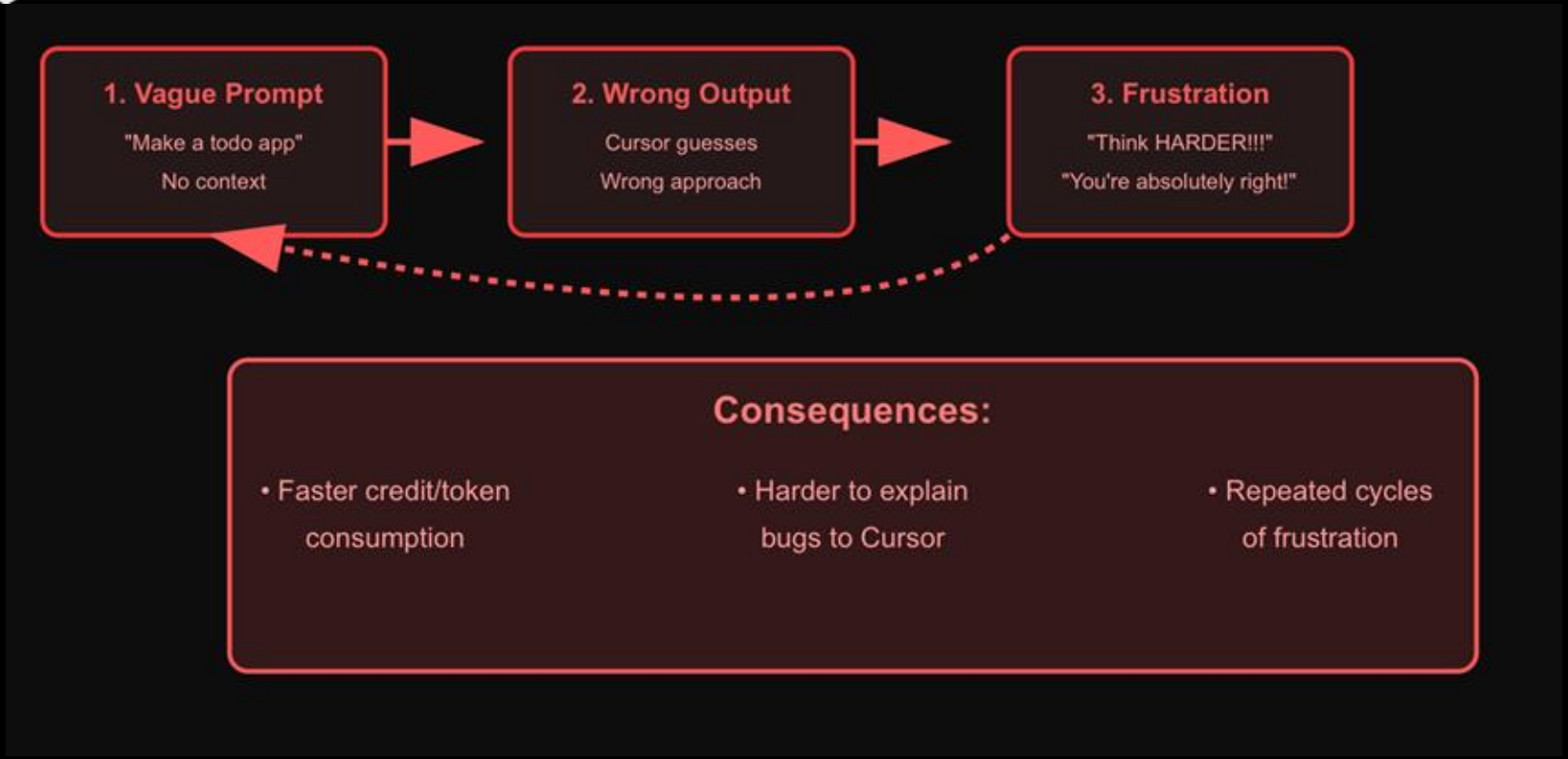
Review – What Humans Must Verify

- Security, intent, performance
- Fix .cursorrules/AGENTS.md when patterns repeat

Own – What Humans Decide

- Architecture, ship decisions, team standards
- AGENTS.md maintenance

Phase 1: Plan - The Problem:



Phase 1: Plan - The Problem:

❌ Bad Prompt

"Make a todo app"

Vague, no context
No structure
Result: "You're right!"

Problems:

- No clear goal
- Missing context
- No architecture

✅ Good Prompt

"Create expense tracker:

1. OVERVIEW.md - project purpose
2. GOAL.md - MVP features
3. ARCHITECTURE.md - Next.js, Tailwind, Zustand
4. JOURNAL.md - task list"

Result: Clear, actionable!

✓ Clear structure
✓ Context provided

Good prompts: Clear goal + Structure + Context + Tech stack

Phase 1: Plan - The Problem:

❌ Bad

```
"Make a todo app"
```

Vague, no context, no structure

✅ Good

```
"Create expense tracker:
1. OVERVIEW.md – project
purpose
2. GOAL.md – MVP features
3. ARCHITECTURE.md –
Next.js, Tailwind, Zustand
4. JOURNAL.md – task list"
```

Clear structure, context, tech stack

Phase 1: Plan - Delegate to AI

Large changes (PRD / roadmap)

- Ingest PRD/roadmap and draft PLAN.md
- Build dependency map across code + infra
- Propose epics and sub-epics with sequencing

Smaller changes (single-issue tasks)

- Generate acceptance criteria from one-liner ticket
- Create 3-5 step mini-plan

Bug fixes / Linear

- Pull in logs/stack traces via Linear MCP
- Suggest reproduction steps and root-cause files

Phase 1: Plan - Review and Own

Engineer Reviews:

- Sanity-check PLAN.md: scope, dependencies, estimates
- Validate all relevant systems are considered
- Remove speculative tasks AI added
- Ensure outputs format for Linear/Jira

Engineer Owns:

- Product vision and prioritization
- Canonical templates (what every plan must include)
- Planning cadence (when to run AI planning)
- Final sign-off that plan is ready

Phase 2: Design - Delegate to AI

Libraries / ShadCN

- Suggest right components and generate skeleton code
- Propose component compositions for common patterns
- Scan repo for custom components to replace

Figma MCP

- Pull tokens (colors, spacing, typography) from Figma
- Generate code stubs from Figma frames
- Detect design-implementation drift

Cursor Browser div-selector

- Capture DOM snippets to identify components
- Suggest CSS/layout tweaks to match Figma

Phase 2: Design - Review and Own

Engineer Reviews:

- Approve library/ShadCN choices for design system
- Evaluate for performance, responsiveness, accessibility
- Validate Figma-to-code mappings
- Ensure Browser tweaks applied in source components

Engineer Owns:

- The design system itself: tokens, taxonomy, patterns
- Which UI libraries are blessed (ShadCN vs others)
- Cross-platform consistency decisions
- Teaching how to brief agents in design terms

Phase 3: Build and Deploy - Delegate to AI

Libraries / Docker and SST

- Generate Infrastructure from Code (sst.config.ts)
- Structure for multi-stage builds to keep images light
- Auto-generate build scripts from package.json

GitHub and Cloud MCPs

- Search PRs and Actions logs to diagnose CI failures
- Check live deployment status and tail logs
- Generate migration SQL comparing local vs prod schema

Cursor Terminal

- Pipe build errors directly into chat (cmd+shift+L)
- Generate curl commands to smoke-test endpoints

Phase 3: Build and Deploy - Review/Own

Engineer Reviews:

- Secrets Audit: No .env values in Dockerfiles/commits
- IAM and Permissions: Dial back to least-privilege
- Cost Sanity: No expensive always-on instances
- Drift Detection: Tweaks in source IaC files, not console

Engineer Owns:

- The Architecture: Monolith vs Microservices vs Serverless
- Production Gating: When to ship
- Security Policy: Who has access, encryption keys
- The Bill: Billing alerts and cost ceilings

Phase 4: Test - Delegate to AI

Libraries / Playwright and Vitest

- Generate Playwright scripts from user flow descriptions
- Ask for resilient locators (getByRole, data-testid)
- Generate edge-case tests: invalid input, network failure

Postgres and Puppeteer MCPs

- Inspect database state before/after test runs
- Let agent visit localhost via Puppeteer to debug failures

Cursor Terminal and Self-Correction

- Run tests via command line
- Highlight stack trace, hit Cmd+K to fix

Phase 4: Test - Review and Own

Engineer Reviews:

- Mock Verification: Reject tests that mock too much
- Flake Detection: Delete hardcoded waitForTimeout()
- Happy Path Bias: Ensure negative test cases exist
- Selector Durability: Use accessibility-based selectors

Engineer Owns:

- The Strategy: Testing Pyramid (Unit vs Integration vs E2E)
- Data Seeding: The Golden Data set for test environment
- Visual Regression: What looks wrong vs pixel shift
- CI/CD Integration: When tests run (blocking vs nightly)

Phase 5: Review - Delegate to AI

AGENTS.md - The Constitution for AI

- Single most important file in your repository
- Hardcode team Must Haves (use Zod, never use any)
- Composer aligns code with rules as it's written

Composer Agent (Cmd+I)

- First-pass reviewer processing entire multi-file context
- Review @Diff for security, performance, .cursorrules
- Agent applies fixes directly (accept/reject inline)

GitHub MCP

- Pull context from related Issues or old PRs
- Check current @Diff doesn't regress previous bug fixes

Phase 5: Review - Review and Own

Engineer Reviews:

- Intent Verification: Does it solve the user's problem?
- Rule Auditing: Same comment twice? Fix AGENTS.md
- Agent Drift: Watch for Lazy Agent mocking complex logic
- Verify Simulated tests actually touch the database

Engineer Owns:

- AGENTS.md maintenance
- Risk Acceptance: Ship it anyway or not?
- Architectural Fit: DRY check at module level
- Teaching: LGTM means validated logic, not just syntax

Phase 6: Document - Delegate to AI

AGENTS.md Generation

- Scan repo: frameworks, folder structure, conventions
- Propose sections per team/client
- How to use Linear, MCP servers, what to avoid

Changelog Management

- After each change, draft changelog entry
- Auto-populate from merged PRs and commit messages

Context Recovery

- Summarize AGENTS.md + changelog at start of new chat
- Agent regains context lost across sessions

Phase 6: Document - Review and Own

Engineer Reviews:

- Curate AGENTS.md for how team actually wants AI to behave
- Check changelog entries are accurate and linked to PRs
- Periodically clean up obsolete instructions
- Keep docs lightweight so people actually read them

Engineer Owns:

- Structure and contract for these docs
- Policy for when updates are mandatory
- Mechanisms agents use to read these files
- Version control, code review, definition of done

Phase 7: Deploy/Maintain - Delegate

Libraries / SST and Docker

- Stop clicking in AWS console
- Define infrastructure strictly in TypeScript
- Refactor Dockerfile for multi-stage builds

MCP / Cloud and Observability

- AWS MCP: Tail CloudWatch logs for deployment analysis
- Sentry MCP: Query top recurring errors, propose fixes

Docs and Runbooks

- Auto-update README after successful deploy
- Write Post-Mortem draft from git diff and terminal logs

Phase 7: Deploy/Maintain - Review/Own

Engineer Reviews:

- Secrets Audit: Verify process.env used, no keys in git (CI/CD)
- IAM Policy: Reject AdministratorAccess, enforce least privilege
- Billing Sanity: Verify instance types and scaling limits
- Destructive Acts: Manually verify DROP, DELETE, FLUSH

Engineer Owns:

- The Pager: Define alerting thresholds (you wake up at 3 AM)
- Feature Flags: Toggle for 10% vs 100% of users
- Data Sovereignty: Where user data lives (GDPR)
- The Stop Button: Authority to freeze deployments

Break

Stretch, grab coffee, ask questions

After the break:

- The Codebase: Aegistry
- Feature: Risk Score Audit Trail
- Hands-on Lab: Plan Phase
- Hands-on Lab: Build Phase

Model Selection Strategy

Planning Phase

Claude Opus 4.5

Deep reasoning
Complex thinking

- ✓ Architecture decisions
- ✓ Multi-file changes
- ✓ Complex refactoring
- ✓ Plan mode

Execution Phase

Composer 1

Fast execution
Cost efficient

- ✓ Implement features
- ✓ Simple edits
- ✓ Quick fixes
- ✓ Code generation

Switch models based on task complexity - Save tokens, get better results!

Key Takeaways

- Context is everything - documentation-first workflow
- Save tokens: `.cursorignore`, `@file`, choose right model
- Planning: Claude Opus 4.5 | Execution: Composer 1
- Use RULES.md to guide Cursor
- Better prompts = Better results (less "Think HARDER!!!1")

The Codebase: Aegistry

- **Banks screen every customer against sanctions lists**
- **Current solutions:**
  - $0.01-0.05 per API call (adds up fast)
  - Black box scoring (compliance officers hate it)
  - US-hosted data (GDPR nightmare)
- **Our mission**: Build a transparent, EU-first alternative.

Aegistry: Our Solution

Own database from public sources

- No external API calls during screening
- Fast and reliable
- Explainable scoring (compliance officers understand flags)
- EU-first (all data in EU regions)
- Current coverage: 26,000+ subjects
  - EU, UN, OFAC, UK sanctions lists
  - Politically Exposed Persons (PEP) lists

Aegistry: Tech Stack

Component	Technology	Why
Language	Rust	Performance + safety
HTTP	Axum	Modern async framework
Database	SQLite	Local, embedded, fast
Search	Tantivy	Rust's Lucene
Matching	Jaro-Winkler	Fuzzy name scoring

Aegistry: Structure

```
aegistry/
  crates/
    core/            # Shared types and DTOs
    matching-core/   # Name matching logic
    ingest/          # Data fetching from lists
    screening-api/   # Axum HTTP API (OUR FOCUS)
  data/
    aegistry.db      # SQLite (~26k subjects)
    index/           # Tantivy search index
  docs/
    blueprint.md     # Product requirements
    changelog.md     # Progress tracking
```

Feature: Risk Score Audit Trail

The Problem:

- Compliance officers need to know WHY someone was flagged
- HOW the score was calculated
- WHEN the screening happened
- WHO made the request
- Currently: We return scores but don't persist reasoning

The solution:

- Store every screening request/response with:
- Score component breakdown
- Which list(s) triggered the hit
- Timestamp and API key

Feature: Acceptance Criteria

#	Requirement
1	Every screening request creates an audit entry
2	GET /v1/audit returns paginated list
3	GET /v1/audit/:request_id returns single entry
4	Tenant isolation: can only see own audits
5	Response includes: request, response, timing, scores

Cursor Workshop

Forta Solutions

January 29, 2025

Agenda

1. **Finish planning, building and testing Aegistry**
2. **Workflow Enhancements**
   a. Figma MCP
   b. Debug Mode
   c. Context Fatigue/Rot
   d. Multi-root workspaces
3. **Agentic Team Collaboration**
   a. AGENTS.md
   b. CHANGELOG.md vs. Git History
   c. Testing

Agenda (cont.)

4. **Browser & Testing Automation**
   - QA
   - Chrome DevTools vs. Playwright vs. Browser-Tools
5. **Command, Hooks & Skills** (Cursor 2.4)
   - Custom Commands + Advanced Patterns
   - Custom Hooks + Advanced Patterns
   - Custom Skills + Advanced Patterns
6. **Implementation Checklist**
7. **Team Implementation Plan**
8. **Recommended Action Items**

Cursor Workshop

https://github.com/cursorworkshop/aegistry.git

January 29, 2025

Hands-on Lab: Setup

1. Go to https://github.com/cursorworkshop/aegistry
2. Fork it to your account
3. Clone in Cursor: File > New Window from Git Repository
4. Health check (commands in chat)

```bash
# Verify project builds
cargo check --all

# Verify API runs
cargo run -p screening-api &

# Test screening endpoint
curl -X POST http://127.0.0.1:3100/v1/persons/screen \
  -H "Content-Type: application/json" \
  -H "X-Api-Key: test-api-key" \
  -d '{"first_name": "Vladimir", "last_name": "Putin", "country": "RU"}'
```

Plan Phase: Activate Plan Mode

1. **Open Composer**
2. **Shift+Tab to enter Plan Mode**
3. **Paste the planning prompt (shared in chat)**
4. **Include context files with @...**

What Plan Mode produces:

- Mermaid diagram of data flow
- Database schema
- API endpoint definitions
- Actionable todo list

Plan: Review and Refine

Review the AI's plan:

- Is the schema complete?
- Any missing indexes?
- Edge cases (no hits, errors)?

The plan is a contract. Don't start building until you're happy with it.

Refine with follow-up prompt:

- "Add to the plan:
  - How to handle screening errors (should we still create an audit entry?)
  - What if payload is very large?"

Plan Phase: Expected Output

Database Schema:

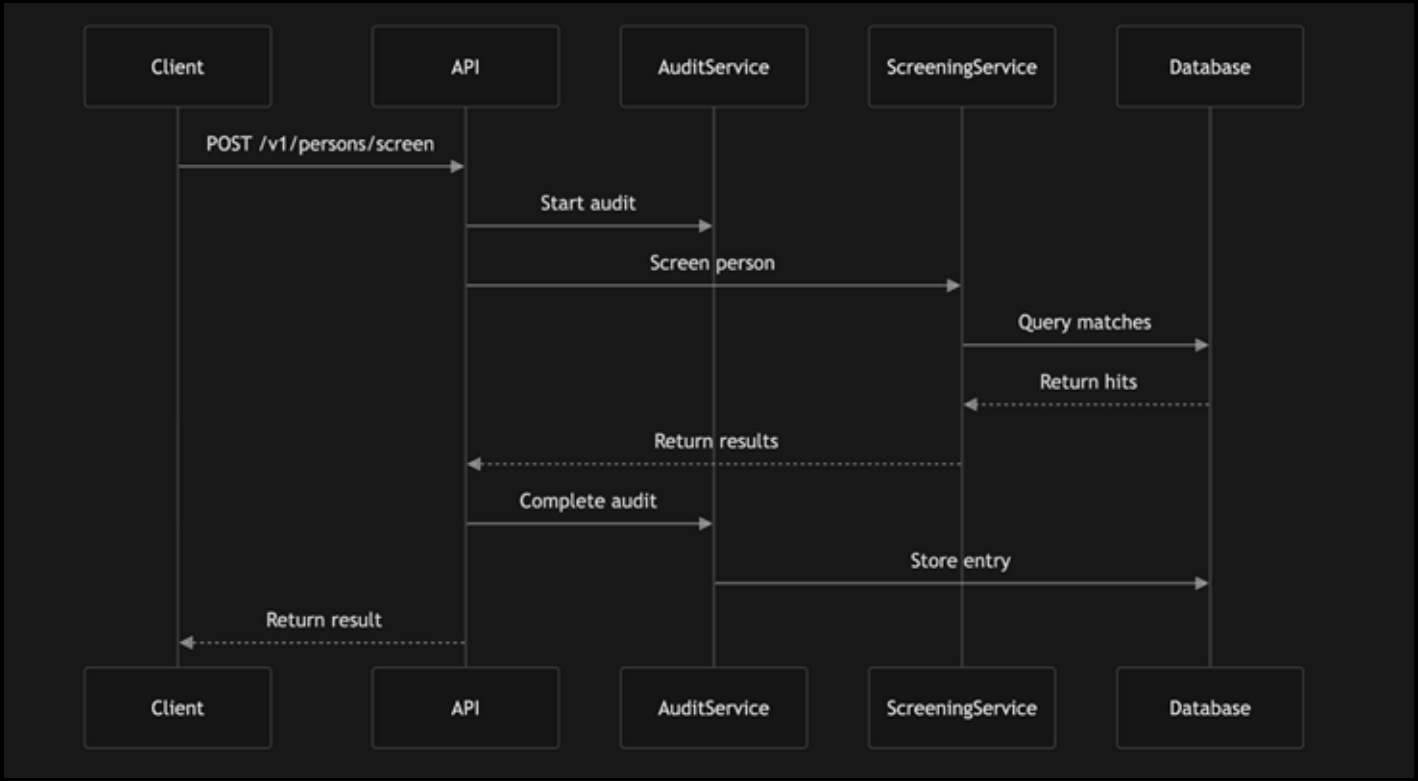
- audit_log table with: id, tenant_id, request_id
- request_payload, response_payload, hit_count, max_score
- processing_time_ms, created_at

API Endpoints:

- GET /v1/audit - List entries (paginated)
- GET /v1/audit/:request_id - Single entry

Mermaid diagram showing data flow (-->)

Plan Phase: Mermaid diagram



Build Step 1: Database Schema

Add audit_log table to tenant_db.rs

Functions to create

1. create_audit_log_table()
2. insert_audit_entry(...)
3. get_audit_entries(tenant_id, limit, offset, from_date, to_date)
4. get_audit_entry_by_request_id(tenant_id, request_id)

Review checklist:

- Uses ? placeholders (SQL injection prevention)
- Returns Result<T, String>
- UUID for IDs, ISO 8601 for dates

Build Step 2: Audit Module

Create audit.rs with structs and handlers

Structs:

1. AuditEntry (matches DB schema, serde)
2. AuditListQuery (limit, offset, dates)
3. AuditListResponse (entries + total_count)

Handlers:

- list_audit_entries: GET /v1/audit
- get_audit_entry: GET /v1/audit/:request_id

Build Step 3: Integrate with Screening

Modify person screening to create audit entries

Flow:

1. Record start_time (std::time::Instant)
2. Run screening as normal
3. Calculate processing_time_ms
4. Generate request_id (UUID)
5. Serialize request + response to JSON
6. Call insert_audit_entry
7. Return response (audit is transparent)

Important: If audit fails, log error but still return screening result!

Build Step 4: Wire up Routes

Add routes to main.rs

Routes:

- GET /v1/audit -> audit::list_audit_entries
- GET /v1/audit/:request_id -> audit::get_audit_entry

Also:

- Protect with auth middleware
- Call create_audit_log_table() on startup

Build Step 5: Succes?

1. **Check compilation**
2. **Run the server**
3. **Test screening**
4. **Check audit entries**

Celebrate?

Cursor Workshop

Part 1: Workflow Enhancements

January 29, 2025

Figma MCP Integration: Design-to-Code Revolution

Transform Figma designs into functional code in minutes

- No more manual design translation or pixel-pushing
- AI reads structured design data: accessibility tree, components, tokens

Three integration options available (Official, Talk-to-Figma, Framelink)

Beta since June 2025, remote server support added September 2025

Figma MCP:

Three Integration Options

- Official Figma MCP Server - Read-only, production-ready, Figma-maintained
- Talk-to-Figma (by Grab) - Bidirectional: read AND modify designs
- Framelink MCP - Read-only, context-optimized for large design files

Choice depends on workflow: generation-only vs. design sync

Official server has rate limits: 6 calls/month (Starter) or per-minute (Dev seats)

Figma MCP: Setup

https://cursor.com/docs/context/mcp/directory

Figma MCP: Setup

Context

MCP Directory

Browse featured MCP servers for Cursor. Use search and filters to find integrations quickly.

Figma

Filters

Reset

Search: Figma ×

Showing 1 of 70

Name	Install	Description
Figma	+ Add to Cursor	Design and collaboration platform for teams.

Don't see your MCP server? Open an issue to request it.

Figma MCP:

get_design_context
Extract design data from selected frame

get_variables_and_styles
Retrieve design tokens (colors, spacing)

create_design_system_rules
Generate translation rules for consistency

Workflow: Select frame in Figma → Prompt Cursor → Get pixel-perfect code

Pro tip: Include external library URLs in prompts for UI frameworks

Figma MCP: Best Practices

- Prepare Figma files with semantic layer names and auto-layout
- Use linked Code Connect mappings for component consistency
- Specify framework (React/Vue), styling (Tailwind/CSS), output paths
- Avoid selecting large frames that exceed context limits
- Design systems with component variants work best for reusability

Live Demo: Batch Component Override Propagation

Propagate design changes across 50+ component instances in seconds

The Problem

Designer updates Button component with new hover state. You have 47 instances across 12 pages that need the same override applied.

Manual: 25+ min → MCP: 8 seconds

Performance Impact

Manual (click each instance)	~15 min
MCP Batch Operation	~8 sec
Time Saved	**112x faster**

Live Implementation

```
// Step 1: Extract source overrides
const sourceOverrides = awaitgetInstanceOverrides ({
  instanceNodeId: "1:23" // Source instance
});
// Step 2: Apply to multiple targets
const result = awaitsetInstanceOverrides ({
  sourceInstanceId: "1:23" ,
  targetNodeIds: [ "1:24" , "1:25" , "1:26" , /* ... 47 total */ ],
  batchSize: 10 // Process in chunks
});
// Step 3: Verify results
console. log ( `Applied: ${result.successCount} / ${result.totalCount} ` );
```

Pro tip: Use batchSize to avoid API rate limits

Override Types Supported: Text content, fill colors, dimensions, visibility, layout spacing. Each property type uses the appropriate MCP tool for atomic updates.

ADVANCED FIGMA MCP

Design Tokens to Code Automation

Extract Figma Variables and generate CSS custom properties automatically

The Token Pipeline

1. **Figma Variables**
   Define colors, spacing, typography in Figma using Variables
2. **MCP Extraction**
   Use get_variable_defs to pull all tokens with metadata
3. **Token Transformation**
   Convert Figma format to CSS/SCSS/JSON output
4. **Code Generation**
   AI generates CSS custom properties with proper naming

Synchronization Strategy

- Design updates variable →
- Run token extraction script →
- AI detects changes & regenerates →
- **CSS updated automatically** ✓

MCP Command

```
// Extract all design tokens
const tokens = awaitget_variable_defs ({
    fileKey: "abc123" ,
    includeValues: true ,
    format: "css" // css, scss, json
});
```

```
// Generated CSS Output
:root {
  --color-primary-500 : #3b82f6 ;
  --color-surface-base : #ffffff ;
  --spacing-md : 16px ;
  --radius-lg : 12px ;
  --font-heading-size : 24px ;
}
```

Desktop MCP server only - requires Figma Desktop app

ADVANCED FIGMA MCP

Code Connect Advanced Workflow

Use actual codebase components instead of generating new ones

What is Code Connect?

Bridge between Figma components and your actual codebase. When AI generates code, it imports YOUR components instead of creating new implementations.

Without Code Connect
Generates new code

With Code Connect
Imports YOUR components

The AI Decision Flow

1. AI calls get_code for the selected frame
2. AI calls get_code_connect_map (Desktop only)
3. Checks: Does Button (node 123:456) have mapping?
4. **YES:** Import from '@/components/ui/Button'
5. **NO:** Generate implementation from scratch

Live Comparison

```
// WITHOUT Code Connect
// AI generates 15 lines of button CSS
export function
  ProductCard () {
  return (
    < divclassName= "button-primary" >
      Add to Cart
    div >
  );
}
```

```
// WITH Code Connect
// AI imports YOUR component
import
  { Button } from'@/components/ui/Button' ;
export function
  ProductCard () {
  return (
    < Buttonvariant= "primary" >
      Add to Cart
    Button >
  );
}
```

ADVANCED FIGMA MCP

Live Demo: Multi-Step User Flow Generation

Extract prototype reactions and generate complete interactive workflows

The Scenario

Designer creates login flow in Figma with multiple screens and interactions. You need to generate code that preserves all states and transitions.

- Login screen with form validation
- Success and error states
- Animations and transitions
- Edge cases (empty fields, wrong password)

What AI Generates

Form Structure
Email input, password input, submit button

State Management
Loading, success, error states with proper handling

Validation Logic
Email format, password strength, required fields

Error Handling
Network errors, auth failures, user feedback

Implementation Code

```
// Step 1: Get prototype reactions
const reactions = awaitget_reactions ({
    nodeIds: [ "1:0" ], // Root node
    recursive: true
});
// Step 2: Visualize as FigJam connectors
awaitset_default_connector ({
    connectorId: "1:5" , // Copy from FigJam
    curveStyle: "SMOOTH"
});
// Step 3: Create connections
awaitcreate_connections ({
    reactions: reactions.nodes. flatMap (n => n.reactions),
    offsetX: 20 , offsetY: 20
});
```

Result: Visual Flow Diagram

AI converts prototype interactions into FigJam connector lines, creating a visual user flow diagram that shows all navigation paths and decision points.

Performance Optimization: Chunked Operations

Handle large-scale operations without hitting API rate limits

The Rate Limit Problem

Figma API has rate limits. Processing 100+ nodes in one request will fail. The solution: chunked processing with throttling.

✗	✓
Single batch of 100 Fails with rate limit	7 batches of 15 Success with delays

Performance Metrics

Nodes processed	100+
Chunk size	15 nodes
Delay between chunks	1000ms
Total time	**~8 seconds**

The Chunking Pattern

Chunk Size Strategy

Start with 15 nodes per chunk. Adjust based on operation complexity. Complex operations need smaller chunks.

Delay Calibration

1000ms delay is safe for most operations. Monitor rate limit headers and adjust dynamically if needed.

Error Recovery

Wrap each chunk in try-catch. Log failures and continue processing remaining chunks for partial success.

ADVANCED FIGMA MCP

Pro Tips for Seasoned Users

Advanced techniques that aren't obvious from the documentation

Metadata Before Code

For large designs, call get_metadata first to understand structure before requesting full code generation.

```
// Quick structure scan
const meta = awaitget_metadata({
  nodeIds: ["1:0"]
});

// Then decide which nodes to process
```

Design System Rules

Use create_design_system_rules to generate project guidance that AI follows for all future generations.

```
// Generate rules from your design
const rules = awaitcreate_design_system_rules({
  fileKey: "abc123",
  format: "cursorrules"
});

// Save to .cursorrules for persistence
```

Batch Text Updates

Use set_multiple_text_contents for batch text updates instead of individual calls.

```
// Update 20 text nodes in one call
awaitset_multiple_text_contents({
  updates: textNodes.map(n => ({
    nodeId: n.id,
    text: "New Text",
    style: { fontSize: 16 }
  })),
  batchSize: 10
});
```

Screenshot Verification

Always use get_screenshot alongside code generation to verify visual fidelity.

```
// Generate code + verify visually
const [code, screenshot] = awaitPromise.all([
  get_code({ nodeIds }),
  get_screenshot({ nodeIds })
]);
```

Selection vs Link

Desktop server supports selection-based workflow. Remote server requires links. Know when to use each.

Desktop (Selection)
Select in Figma → Prompt Cursor. Faster for iterative work.

Remote (Link)
Copy link → Paste in Cursor. Better for sharing/specific frames.

Prototype Visualization

Convert prototype reactions to FigJam connectors for visual user flow diagrams. Great for documentation.

```
// Visualize user flows
const reactions = awaitget_reactions({
  nodeIds: ["1:0"],
  recursive: true
});

// Then create_connections for FigJam
```

Debug Mode: AI Meets Runtime Reality

- Released December 2025 in Cursor 2.2
- Entirely new agent loop built around runtime information
- Hypothesis-driven debugging with human verification
- Solves: AI fixes that 'look right' but don't actually work
- Cross-stack support: Java, TypeScript, Python, Go confirmed

The Problem Debug Mode Solves

- Traditional AI debugging: Guesses based on code reading alone
- Models can't see actual execution state, variable values, timing
- Result: Multiple fix attempts, frustration, time wasted
- Debug Mode forces systematic approach with evidence
- Runtime data validates or rejects hypotheses before code changes

Debug Mode: How it Works

1. Describe the bug → Agent generates 5-7 hypotheses
2. Agent narrows to 1-2 most likely causes
3. Agent instruments code with targeted logging
4. You reproduce the bug → Logs captured to .cursor/debug.log
5. Agent analyzes runtime data → Proposes targeted fix
6. You verify → Click 'Mark Fixed' → Instrumentation removed

Debug Mode: Log Output Format

```
{
  "location": "DemoApplication.java:70",
  "message": "Extracted values",
  "data": {
    "dbStatus": "ACTIVE",
    "categoryInput": "FOOD",
    "timestamp": "2026-01-15T10:23:45Z"
  },
  "hypothesisId": "A,B,C"
}

// Each log entry ties back to specific hypotheses
// Agent uses this to validate or reject each theory

// Example: If hypothesisId "A" expected dbStatus
// to be "PENDING" but it's "ACTIVE", hypothesis A
// is rejected and agent moves to hypothesis B
```

Debug Mode: When to use it

Use Debug Mode

- Runtime/state bugs
- Race conditions
- API integration issues
- “It works in tests but fails live”
- Intermittent failures
- Complex state interactions

Use Regular Agent

- Feature implementation
- Refactoring
- Code generation
- Documentation
- Test writing
- Straightforward fixes

Debug Mode: Java-Specific Considerations

- Build/restart cycle slower than interpreted languages
- Agent doesn't auto-restart yet - manual step required
- Pro tip: Keep build commands ready (./gradlew build)
- Consider hot-reload tools (JRebel, Spring DevTools)
- Requires reproducible bugs - doesn't work for flaky issues

Context Rot

- Question: 'Do agents get fatigued?' - Answer: Yes!
- AI has limited 'working memory' (context window)
- As conversation grows: Instructions forgotten, repetitive mistakes
- Signs: Agent repeating itself, ignoring early instructions
- January 2026: Dynamic Context Discovery reduced tokens by 46.9%

What is Context Window Fatigue?

- Default chat mode: ~20,000 tokens of context
- Max Mode: 200K tokens with Claude 4 Sonnet/Opus (~$0.05/request)
- When limits reached: Automatic summarization (lossy compression)
- Result: 'Context amnesia' - agent forgets earlier progress
- Symptoms: Suggesting already-rejected solutions, re-reading docs

How Cursor Handles Context

- Automatic summarization when context fills up
- Summary becomes new context - but it's lossy compression
- History files available for agent to search back
- Dynamic Context Discovery: On-demand loading, not upfront dump
- Long tool responses now write to files for selective retrieval

Manual Refresh Strategies

- Fresh Chat (Cmd+N / Ctrl+N) - Clean slate, fastest fix
- Manual Summarize - Copy key points, paste into new chat
- Checkpoint files - Keep .cursor/scratchpad.md with current state
- Plan files - Save plans to .cursor/plans/ for resumption
- Use /summarize or /compress commands to manually trigger

Pro Tips for Long Sessions

- Keep conversations focused: one feature/bug per chat
- Restart after completing a feature
- Use @file references instead of pasting code
- Document project state in README/AGENTS.md for quick reload
- Be surgical with @ mentions - specific files, not entire folders

Multi-Root Workspaces

- Shipped in Cursor 0.50 (May 2025)
- Multiple repositories in a single workspace
- All folders added are indexed
- AGENTS.md & .cursor/rules files work across roots
- Important for: Java microservices + DevOps code together

Multi-Root: Current Limitations

- Cursor defaults to FIRST root folder in workspace file
- @diff and @branch only pass git changes from first project
- AI can create incorrect paths (absolute vs relative)
- May fail to maintain awareness of different tech stacks
- Multi-Agents functionality NOT supported in multi-root

Multi-Root: Workarounds

- Place your primary repository FIRST in workspace file
- Be explicit when prompting: 'Focus on apps/serviceA for this task'
- Use explicit @foldername/path/file syntax for cross-root references
- Consider git worktrees for complex workflows
- Community tools like Repomix for comprehensive codebase views (or Repofy, open-source project by me [Rogier])

Multi-Root: What's Coming

- Better multi-root support actively in development
- Independent context isolation per project (planned)
- Automatic AI context switching based on active file
- Multi-agent support for multi-root (requested feature)
- For now: Scope agents explicitly to folders when prompting

Cursor Workshop

Part 2: Team Collaboration Patterns

January 29, 2025

AGENTS.md: README for Your AI Teammates

- One file, any agent: Cursor, Claude Code, Codex, Copilot
- Solves fragmentation: .cursorrules, .windsurfrules, CLAUDE.md
- Now in 60,000+ GitHub repositories (up from 20K in early 2025)
- Stewarded by Agentic AI Foundation (Linux Foundation)
- Collaborative development: OpenAI, Google, Cursor, Sourcegraph

AGENTS.md: Recommended Structure

```
# AGENTS.md

## Tech Stack
- Java 21, Spring Boot 3.2, Gradle
- PostgreSQL 16, Redis for caching

## Build & Test
- Build: ./gradlew build
- Test: ./gradlew test
- Local run: ./gradlew bootRun

## Project Structure
- /apps - Microservices
- /libs - Shared libraries
- /infra - Terraform, K8s configs

## Code Style
- Follow existing patterns in src/main/
- Use Lombok for boilerplate
- Require tests for new features

## Off-Limits (Never modify without review)
- config/production.yml
- secrets/ directory
- Database migrations (require DBA approval)
```

AGENTS.md: README for Your AI Teammates

- Use hierarchical AGENTS.md files - nearest file wins
- Root AGENTS.md for global rules and org standards
- Nested AGENTS.md per package for specific conventions
- Example: OpenAI's main repo uses 88 AGENTS.md files
- Agents automatically read the nearest file in directory tree

AGENTS.md: Tool Compatibility

Tool	Support	Notes
OpenAI Codex	Native	Primary driver of standard
GitHub Copilot	Native	Uses .github/agents/*.md
Cursor	Native	Also supports .cursorrules
Google Jules/Antigravity	Native	Configure via .gemini/settings.json
Claude Code	Via CLAUDE.md	Symlink recommended
Windsurf, Devin, Aider	Native	Full support

AGENTS.md: Best Practices

- Be explicit and concise - AI reads literally
- Keep instructions current - outdated rules cause confusion
- Link to existing docs (don't duplicate content)
- Include what you'd tell a new teammate on day one
- Don't over-engineer or include secrets

AGENTS.md: Syncing Across Tools

```
# From your repo root:

# For Claude Code
ln -s AGENTS.md CLAUDE.md

# For GitHub Copilot
mkdir -p .github
ln -s ../AGENTS.md .github/copilot-instructions.md

# Verify symlinks
ls -la CLAUDE.md .github/copilot-instructions.md

# Sync script (optional)
#!/bin/bash
# save as sync-agents.sh
cp AGENTS.md CLAUDE.md
cp AGENTS.md .github/copilot-instructions.md
echo "Synced AGENTS.md to all tools"

# Single source of truth - edit only AGENTS.md
```

CHANGELOG.md in Team Setting

- Problem: Manual changelogs get deprioritized and forgotten
- Solution: AI-assisted automation from git history
- Foundation: Conventional Commits enable auto-categorization
- Provides rich context for AI agents about recent changes
- Bridges communication between developers and stakeholders

CHANGELOG.md vs Git History: Why Both?

Audience	Why CHANGELOG.md beats raw git history
Humans	Curated narrative - no WIP commits, typos, or merge noise
	Non-dev accessible - PMs/stakeholders don't need git
	Survives rebases - git history gets rewritten, CHANGELOG doesn't
AI Agents	Token-efficient - 10x smaller than git log output
	No CLI required - just Read CHANGELOG.md, no git commands
	Structured = parseable - ### Added, ### Fixed sections easy to query
	Fast session bootstrap - "what changed?" is one file read
Teams	Shared definition of "notable" - team agrees what counts
	Release-aligned - changes grouped by version
	Async onboarding - new members (human or AI) get context